Studio Nx

JS 112 — Metronome 클래스

2026-10-14

JS 112 — `Metronome` 클래스 · `countInTicks()` (메트로놈 on/off·카운트인)

항목	내용
파일	`studionx/frontend/static/js/audio/metronome.js` (111에 이어 추가)
작업	S1-05 오디오 엔진 기초: 아홉 번째 함수 (FR-203 메트로놈 on/off·카운트인)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	루프(S1-05 마지막)

1. 하는 일

111의 순수 판정(`metronomeClick`) 위에 on/off와 클릭 소리 연결을 얹었습니다. 소리는 페이지가 넘긴 `click(time, accent)` 콜백에서 냅니다(AudioContext는 페이지 담당). 겸해서 카운트인 길이를 돌려주는 `countInTicks()`를 더했습니다.

대상	하는 일
`new Metronome({click, enabled?, timeSigNum?, timeSigDen?})`	`click` 콜백 필수(없으면 `TypeError`). 기본은 꺼짐·4/4
`setEnabled(on)`	메트로놈 켜기/끄기 (FR-203)
`setTimeSignature(num, den)`	박자 바꾸기 (FR-202a)
`step(tick, time)`	켜져 있고 박 위이면 `click(time, accent)`를 부르고 `{beatIndex, accent}` 반환, 아니면 `null`
`countInTicks(num, den)`	한 마디 길이(tick). 트랜스포트를 그만큼 앞에서 시작하면 카운트인이 들림

페이지 사용 예(개념):

```
const metronome = new Metronome({ click: playClick, enabled: true, timeSigNum: 4, timeSigDen: 4 });
transport.onStep = function (tick, time) { metronome.step(tick, time); };
transport.start(-countInTicks(4, 4));  // 한 마디 카운트인 뒤 0 에서 곡 시작
```

2. 코드

```
function assertTimeSig(timeSigNum, timeSigDen) {
  if (!Number.isInteger(timeSigNum) || timeSigNum < 1 || timeSigNum > 32) throw new RangeError('timeSigNum 은 1~32 정수여야 합니다: ' + timeSigNum);
  if (TIME_SIG_DEN_VALUES.indexOf(timeSigDen) === -1) throw new RangeError('timeSigDen 은 1,2,4,8,16,32 중 하나여야 합니다: ' + timeSigDen);
}

export function countInTicks(timeSigNum, timeSigDen) {
  assertTimeSig(timeSigNum, timeSigDen);
  return ((PPQ * 4) / timeSigDen) * timeSigNum;
```

```
}

export class Metronome {
  constructor(options) {
    options = options || {};
    if (typeof options.click !== 'function') throw new TypeError('click 콜백(함수)이 필요합니다');
    this.click = options.click;
    this.enabled = options.enabled != null ? !!options.enabled : false;
    this.timeSigNum = options.timeSigNum != null ? options.timeSigNum : 4;
    this.timeSigDen = options.timeSigDen != null ? options.timeSigDen : 4;
    assertTimeSig(this.timeSigNum, this.timeSigDen);
  }
  setEnabled(on) { this.enabled = !!on; return this; }
  setTimeSignature(timeSigNum, timeSigDen) { assertTimeSig(timeSigNum, timeSigDen); this.timeSigNum = ti
meSigNum; this.timeSigDen = timeSigDen; return this; }
  step(tick, time) {
    if (!this.enabled) return null;
    const info = metronomeClick(tick, this.timeSigNum, this.timeSigDen);
    if (info === null) return null;
    this.click(time, info.accent);
    return info;
  }
}
```

최신 문법으로 쓰면 (공부용)

```
  constructor({ click, enabled = false, timeSigNum = 4, timeSigDen = 4 } = {}) {
    if (typeof click !== 'function') throw new TypeError('click 콜백(함수)이 필요합니다');
    // …
  }
  setEnabled(on) { this.enabled = Boolean(on); return this; }
```

최신 문법	실제 코드	뜻
구조 분해 기본값 `{ click, enabled = false } = {}`	`options = options	
`Boolean(on)`	`!!on`	값을 참/거짓으로 바꿉니다
선택적 체이닝 `options?.click`	`options.click`	앞이 없을 때만 안전하게 접근합니다(전통 문법에서는 `options || {}` 로 먼저 채움)

3. 왜 이렇게

선택	이유
소리는 콜백으로 주입	클릭 소리(AudioContext)를 파일 밖으로 빼서 Node로 on/off·박 판정을 시험합니다. 소리 내는 부분은 페이지가 담당
`assertTimeSig` 를 따로 둠	111·`countInTicks`·클래스 세 곳이 같은 검사를 씁니다. 한 곳에 모아 규칙이 어긋나지 않게 함
기본 꺼짐	메트로놈은 사용자가 켜야 들립니다(FR-203 on/off). 처음부터 소리가 나면 놀람
꺼져 있으면 바로 `null`	판정 전에 돌아가 불필요한 계산을 줄입니다
`step` 이 판정 결과를 돌려줌	페이지나 시험에서 "이번 단계에 울렸는지" 확인할 수 있습니다
카운트인은 트랜스포트 시작점	카운트인은 0보다 앞에서 울리는 것이라, 클래스가 소리를 따로 세지 않고 시작 위치로 처리합니다. 음수 tick은 111이 이미 격자로 다룸

선택	이유
`setEnabled`·`setTimeSignature` 가 self 반환	잇달아 부르기 쉽게 합니다(`m.setEnabled(true).setTimeSignature(6,8)`). 110 `setTempo` 와 같은 방식

4. 시험 (Node에서 직접 실행, 31개 통과 · 클릭 콜백 기록)

확인	결과
`countInTicks` 4/4=3840, 6/8·3/4=2880, 7/8=3360, 오류 시 `RangeError`	✓
`click` 콜백 없으면 `TypeError`	✓
기본 꺼짐·4/4, 꺼진 채 `step` 은 `null`·클릭 없음	✓
켜면 강박·박에서 클릭, `time`·`accent` 전달, 박 사이는 `null`	✓
끄면 다시 안 울림, `setEnabled` self 반환	✓
`setTimeSignature(6,8)` 로 박 격자 변경(8분음표 480), self 반환, 오류 시 `RangeError`	✓
생성 시 박자 지정(3/4) 반영	✓
111 `metronomeClick` 회귀	✓